SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

B. Tech. Project Poster AY 2026-27

TITLE: ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

Faheemuddin Sayyed (23070122196) | **Sanidhya Awasthi** (23070122192) | **Raghav Sonchhatra** (23070122172)

Guide Name: Dr. Payal Kadam

Celebrating 50 Years of Excellence

INTRODUCTION

An LLM can now generate a web app in minutes. Knowing that the app works, and knowing when to stop fixing it, is still hard.

- Asking a model to keep improving its own code can make the code less secure. (Shukla et al., 2025)
- Fewer than 1 in 3 model-generated infrastructure deployments succeed on the first attempt. (Zhang et al., 2026)
- Most of a repair loop's gain arrives in the first 1–3 rounds; later rounds mostly spend tokens. (Kiecker et al., 2026; Arimbur, 2026)
- Developers steer coding agents closely rather than hand over control. (Huang et al., 2025; Tang et al., 2026)

The gap

	Research repair agents	LLM test / repair tools	Prompt-to-app builders	ForgeFlow
Repair with an iteration cap and stall detection	×	×	×	✓
Hands back to a person when repair stalls	×	×	×	✓
Requirements compiled into traceable tests	×	*partial*	×	✓
Automated cloud deployment	×	×	✓	✓
Re-tests the deployed, live URL	×	×	×	✓
Any stage can be skipped or revisited	×	×	*partial*	✓

Capabilities as described in the cited work, adapted from Table 1 of our paper; not a performance comparison.

ForgeFlow takes a web-app idea, given as screenshots or a structured spec, to a live URL that has been tested, and lets a person enter, refine or skip any stage on the way.

OBJECTIVES & AIMS

01 Non-linear control. No stage is mandatory; every artifact is versioned and can be revisited.

02 Generation. Screenshots or a spec become a full-stack app on one fixed stack.

03 Requirements to tests. Acceptance criteria compile into unit and E2E tests, each traceable to its criterion.

04 Bounded self-healing repair. A diff-aware loop with caps, stall detection and escalation to a person. (Core contribution)

05 Infrastructure reasoning. Work out what each app needs, then deploy it.

06 Live validation. Re-test the deployed URL; repair and redeploy on failure.

07 Isolation. Generated code runs only in a per-project sandbox.

08 Measurement. First-pass vs. post-repair success, cost and latency, reproducibly.

Hypotheses under test

H1 Bounded repair captures most of the achievable gain at a fraction of an unbounded loop's cost.

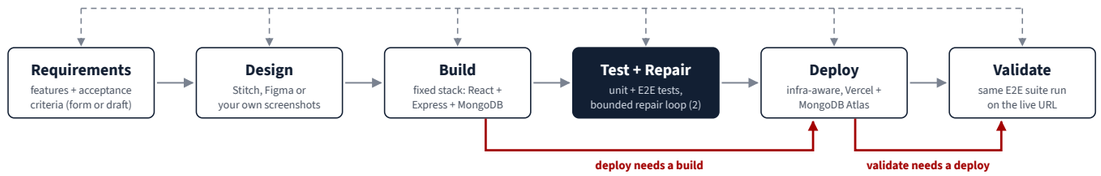
H2 A minimal, diff-aware context repairs more per token than broad repository context.

H3 Re-testing the live URL catches faults that sandbox tests miss.

METHODOLOGY

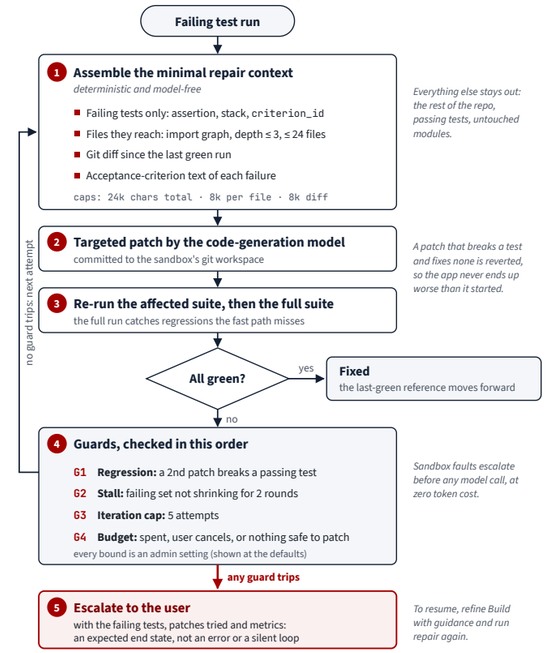
1 Six stages, none mandatory — The conductor, not the UI, decides which transitions are legal.

You can enter at any stage, skip any stage, or jump back to refine one.

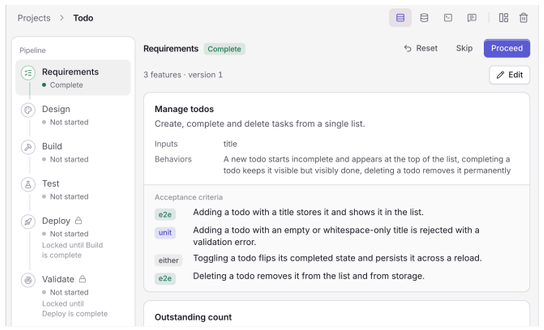


Refine a stage and later stages turn stale. Nothing is deleted; every artifact is versioned.

2 Bounded, diff-aware self-healing repair

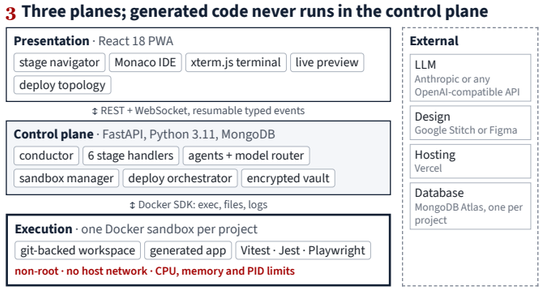


Why a diff: generated tests lose validity when code changes, and highlighting the change is the recommended fix (Haroon et al., 2026). **Why a cap:** most gain comes in the first rounds (Kiecker et al., 2026).



The workspace (real UI). Each acceptance criterion carries a `unit` or `e2e` tag. Deploy and Validate stay locked until Build and Deploy finish; every other stage can be skipped.

3 Three planes; generated code never runs in the control plane



4 One identifier from requirement to repair — An unmet criterion is reported, not averaged into a pass rate.

Acceptance criterion "Deleting a todo removes it from the list and from storage" (e2e) → **Tagged test** each generated test carries the criterion's `criterion_id` → **Normalised result** `{status, assertion, stack, files, criterion_id}` → **Criterion verdict** met only if a tagged test ran and none failed; untested counts as unmet → **Repair context** the failing criterion's text is handed to the patch step (2)

5 Deploy in order, then test the live URL

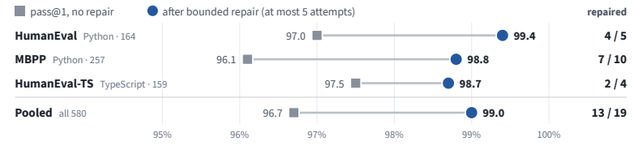
Classify static site, frontend + API, or frontend + API + database → **Provision DB** MongoDB Atlas, one database per project → **Deploy API** Express as a Vercel function, database URI injected → **Deploy UI** static SPA on Vercel, API URL injected → **Health-check** every node, so a dead API never ships behind a live UI → **Validate live** the same Playwright suite, run against the public URL

Live failure → same bounded loop → redeploy → re-validate (at most 2 cycles). Tokens are platform-owned or the user's own, Fernet-encrypted at rest and never sent to the browser.

RESULTS & CONCLUSIONS

One model (GLM-5.3-flash) through ForgeFlow's own model interface; one reproducible run, 2 Oct 2026. Problems from HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021) and HumanEval-TS (Cassano et al., 2023).

Bounded repair on 580 coding problems



The pooled gain is significant (exact McNemar p = 0.00024); per benchmark, only MBPP's is (p = 0.016).

What happened to the 19 first-attempt failures



+5.7% model calls for the repairs (33 on top of 580) — **0** runs reached the cap of 5; the stall guard did the bounding

Tool calling: BFCL v3 — 1,240 cases, official AST checker, no repair (Patil et al., 2025)

92.7% mean of the 4 AST categories — **90.0 – 96.2%** range across the 4 categories — **68.8%** irrelevance: calls no tool

Whole applications — 2 of 10 specs run, once each

78 / 78 unit tests passing — **2** / 13 browser E2E tests passing — **12** / 16 acceptance criteria met

Every unmet criterion was a browser test, which imports no app source, so the loop had nothing to edit and escalated (`blocked`) in about 1.3 s instead of guessing. Neither app was delivered.

Conclusions

- **Repair pays where a failing test reaches the code:** 13 of 19 first-attempt failures fixed for 5.7% more model calls, and the guards, not the cap, ended every unproductive run.
- **It knows when to stop:** all 580 problems and both apps ended with a recorded, typed reason, and each unmet criterion traces to a failing test.

H1 SUPPORTED at function level; one model, one run

H2 NOT YET TESTED context-ablated arm still to run

H3 QUALITATIVE e.g. a database URI that failed only once deployed

Next: map failing browser tests to the pages and routes they visit, so repair can act on them; then run all 10 apps.

REFERENCES (APA)

Arimbur, J. J. (2026). *How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks* (arXiv:2604.10508). arXiv.

Austin, J., Odena, A., Nye, M., et al. (2021). *Program synthesis with large language models* (arXiv:2108.07732). arXiv.

Cassano, F., Gouwar, J., Nguyen, D., et al. (2023). MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering, 49*(7), 3675–3691. https://doi.org/10.1109/TSE.2023.3267446

Chen, M., Tworek, J., Jun, H., et al. (2021). *Evaluating large language models trained on code* (arXiv:2107.03374). arXiv.

Haroon, S., Khan, M. T., & Gulzar, M. A. (2026). *Evaluating LLM-based test generation under software evolution* (arXiv:2603.23443). arXiv.

Huang, R., Reyna, A., Lerner, S., et al. (2025). *Professional software developers don't vibe, they control: AI agent use for coding in 2025* (arXiv:2512.14012). arXiv.

Kiecker, T., Reichmann, E., Kang, H., et al. (2026). *Is three the magic number? An empirical evaluation of LLM-based repair loops* (arXiv:2607.05197). arXiv.

Patil, S. G., Mao, H., Yan, F., et al. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proc. 42nd International Conference on Machine Learning (ICML)*, PMLR 267.

Shukla, S., Joshi, H., & Syed, R. (2025). Security degradation in iterative AI code generation: A systematic analysis of the paradox. In *Proc. 2025 IEEE International Symposium on Technology and Society (ISTAS)* (pp. 1–8). https://doi.org/10.1109/ISTAS65609.2025.11269659

Tang, N., Chen, C., Xu, G., et al. (2026). *How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions* (arXiv:2605.29442). arXiv.

Zhang, T., Pan, S., Zhang, Z., et al. (2026). Deployability-centric infrastructure-as-code generation: Fail, learn, refine, and succeed through LLM-empowered DevOps simulation. In *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*.

ACKNOWLEDGEMENT

We thank our guide, **Dr. Payal Kadam**, whose questions about what the system could actually demonstrate shaped its design and how we report results; **Dr. Sonali Kothari**, Head of the Department of CSE, for facilities and support; and the faculty who reviewed our progress and invention disclosure.

OUTCOME OF THE PROJECT WORK

Springer Nature research paper
Submitted to *Software and Systems Modeling*, 17 Oct 2026; under review.

Invention disclosure
Bounded diff-aware self-healing delivery; submitted for filing.

IBM Bob 2.0 Hackathon
ForgeFlow submitted on lablab.ai, 25–27 Sep 2026; completion certificates for all three members.

Working platform
65 phases, about 54k lines, 2,323 automated tests; all 2,253 run on the dev host pass.

Platform CI/CD
Six-job CI; health-checked deploys. Rollback drill: 807 requests, 0 failed.

TRY IT LIVE
Scan to open the deployed platform.
`forgeflow-ecru.vercel.app`
No industry collaborator on this project.